premium-wedding-invitation 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: "Eternal Promise" 모바일 청첩장. 히어로/갤러리/달력/오시는 길/연락처·계좌/방명록 섹션과 배경음악, 링크·카카오 공유, AI 축하말 생성 기능을 가진 단일 페이지 앱.
- 목적/대상 사용자: 예비 부부(구매자)와 링크를 받는 하객. nexus-micro-web-builder 의 12개 모델 중 0번 "모바일 청첩장"을 실제 상품으로 구현한 프로토타입이다.
- 출처: Google Vertex AI Studio export 구조. backend/server.js, frontend/vertex-ai-proxy-interceptor.js, README.md 가 nexus-micro-web-builder 와 동일 템플릿이다. 단, nexus 와 달리 이 프로젝트는 프론트에서 Vertex AI(Gemini, Imagen)를 실제로 호출하므로 백엔드 프록시가 "미사용 잔재"가 아니라 실행 경로에 있다.

기술 스택

영역	기술	비고
프론트	React 19.2.5, TypeScript 6.0.3, Vite 8.0.10, Tailwind CSS(CDN 런타임)	package.json 전부 `latest`, tsconfig 없음
UI	framer-motion 12, lucide-react, canvas-confetti	
백엔드	Node.js, Express 5, google-auth-library, ws, express-rate-limit	Vertex AI 프록시 템플릿 원본
DB	Supabase(placeholder) / localStorage 폴백	services/supabase.ts URL/키가 더미라 항상 localStorage 모드
인프라	npm workspaces, concurrently	배포 설정 없음, 백엔드는 127.0.0.1 바인딩
AI	@google/genai 1.50 (vertexai 모드) - gemini-2.5-flash, imagen-4.0-generate-001	백엔드 ADC 경유, 브라우저에 API 키 없음

현재 완성도 추정: 약 45%

- 근거(완성): 7개 섹션 컴포넌트가 모두 렌더링되고 계좌 복사, 전화/문자 링크, 네이버/카카오 지도 딥링크, 달력, 음악 토글, 링크 복사가 동작한다.
- 근거(미완): 하객별 데이터 입력/관리 기능이 없고 모든 정보가 constants.ts 하드코딩이다. Supabase 와 카카오 SDK 는 placeholder 라 방명록은 하객 간 공유되지 않으며(localStorage), 카카오 공유는 동작하지 않는다. 사진이 실제 신랑·신부가 아닌 AI 생성/picsum 랜덤 이미지다. 공개 배포 경로가 없다.

핵심 기능 목록

기능	상태
히어로 캐러셀(Imagen 3장 생성)	동작하나 방문마다 생성, 실제 커플 사진 아님
인사말/혼주, 달력, 갤러리	동작(정적 데이터, picsum 이미지)
오시는 길(Imagen 지도 이미지 + 지도 앱 링크)	링크 동작, 지도 이미지는 AI 가 그린 가짜 지도
연락처/계좌 복사/화환 링크	동작
방명록 + AI 축하말(Gemini)	로컬 저장만 동작, Supabase 미설정
카카오 공유	미동작(키 placeholder, SRI 해시 오류)

2. 구조 분석

경로	역할	비고
package.json	루트 workspace, npm run dev 로 프론트+백엔드 동시 실행	concurrently latest
backend/server.js (453줄)	Vertex AI REST/WS 프록시, ADC 인증, IP당 15분 100회 rate limit	nexus 와 동일 템플릿
backend/.env.local	GCP 프로젝트/리전/PROXY_HEADER	루트 .gitignore:20 으로 제외(정상)
frontend/index.tsx	엔트리, 인터셉터 import	
frontend/vertex-ai-proxy-interceptor.js	window.fetch/WebSocket 을 가로채 /api-proxy 로 우회	PROXY_HEADER 하드코딩
frontend/App.tsx (233줄)	레이아웃, 음악, 토스트, 공유, confetti	적정 크기
frontend/components/*.tsx (7개, 각 60~200줄)	섹션별 컴포넌트	거대 컴포넌트 없음
frontend/services/gemini.ts, imageService.ts	Gemini 축하말, Imagen 히어로/지도 생성	브라우저에서 직접 호출
frontend/services/supabase.ts	Supabase 클라이언트(더미 URL이면 null)	하드코딩 키 자리
frontend/constants.ts, types.ts	예식 데이터, 갤러리, 음악 URL	단일 커플 하드코딩

데이터 흐름: 모든 예식 정보는 constants.ts 의 WEDDING_DATA 를 App 이 props 로 각 섹션에 내려준다. Hero 와 Location 은 마운트 시 imageService 를 호출하고, @google/genai 가 aiplatform.googleapis.com 으로 보내는 fetch 를 인터셉터가 가로채 Vite 프록시(/api-proxy) -> 백엔드 -> ADC 토큰 부착 -> Vertex AI 로 전달한다. 방명록은 supabase 가 null 이므로 localStorage 에만 저장된다. 상태는 컴포넌트 로컬 useState 뿐이며 전역 상태관리는 없다(규모상 적절).

nexus-micro-web-builder 와의 공통 문제: 동일 템플릿에서 온 #5(PROXY_HEADER 프론트 하드코딩), #6(클라이언트 헤더가 Authorization 덮어쓰기), #8(node-fetch phantom dependency), #9(latest 버전, tsconfig 없음), Tailwind CDN 이 그대로 반복된다. 차이점은 이 프로젝트가 AI 를 실제로 쓰기 때문에 "백엔드 삭제"가 아니라 "AI 호출을 빌드 타임으로 옮기거나 서버 전용 엔드포인트로 바꾸는 것"이 해법이라는 점이다.

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	성능/비용	frontend/components/Hero.tsx:24-31, frontend/components/Location.tsx:17-23, frontend/services/imageService.ts:16-27,43-52	페이지 방문마다 Imagen 4 이미지 4장(히어로 3 + 지도 1)을 새로 생성. 하객 수백 명이면 수백~수천 건 과금, 첫 화면이 생성 완료까지 수 초간 회색 스켈레톤. StrictMode 개발 환경에서는 2배 호출
2	높음	버그/제품	frontend/services/imageService.ts:11-13,47	히어로가 실제 신랑·신부가 아닌 "임의의 한국인 커플" AI 이미지이고 방문마다 바뀜. 지도는 Imagen 이 그린 가짜 "네이버 지도 스타일" 이미지라 도로/위치가 사실과 다를 수 있어 하객 길안내를 오도
3	높음	구조/설정	backend/server.js:18,313, frontend/vite.config.ts:11-15	AI 호출이 브라우저 -> 로컬 Vite 프록시 -> 127.0.0.1 백엔드 구조라 공개 배포 시 그대로 쓸 수 없음. 배포하려면 백엔드를 공개해야 하고, 그러면 #4, #5 와 결합해 누구나 GCP 크레딧으로 임의 모델을 호출하는 오픈 프록시가 됨
4	중간	보안	backend/server.js:198-237	/api-proxy 가 모델명({{model}})과 body 를 클라이언트가 그대로 지정. 허용 모델/용도 화이트리스트 없음. ReasoningEngine 패턴은 project_id 까지 클라이언트 값 사용(server.js:106)

#	심각도	분류	위치(경로:줄)	내용
5	중간	보안	frontend/vertex-ai-proxy-interceptor.js:116	PROXY_HEADER 시크릿이 프론트 소스에 하드코딩되어 번들에 포함(nexus 공통)
6	중간	보안	backend/server.js:232	headers: {...apiHeaders, ...headers} 로 클라이언트 헤더가 Authorization, X-Goog-User-Project 를 덮어쓸 수 있음(nexus 공통)
7	중간	버그	frontend/components/Guestbook.tsx:53-63,93-97	Supabase 모드에서 insert 오류 무시, relation 미저장, id 를 Date.now() 문자열로 넣어 uuid 컬럼(supabase.ts:7 주석)과 타입 불일치. 삭제 버튼은 누구나 보이며 localStorage 만 수정해 Supabase 모드에선 새로고침 시 복원됨. 이름/내용 길이 제한·검증 없음
8	중간	설정	backend/server.js:10, backend/package.json:13-19	node-fetch import 하지만 dependencies 에 없음(lock 의 전이 의존성 3.3.2 에 의존). nexus 공통
9	중간	설정	frontend/package.json:9-26, package.json:10	모든 의존성 latest, tsconfig.json/@types/react 없음 -> 타입 검사 불가, 재현 불가능한 설치(nexus 공통)
10	중간	버그	frontend/index.html:20, frontend/App.tsx:39	카카오 SDK 의 SRI integrity 값이 가짜 문자열(987u987u...)이라 브라우저가 스크립트 로드를 차단. 키도 YOUR_KAKAO_JS_KEY. 결과적으로 카카오 공유는 항상 navigator.share 폴백
11	낮음	버그	frontend/index.html:24	font-family: 'Noto+Sans+KR' 오타(공백 대신 +). 본문 폰트가 적용되지 않음
12	낮음	보안	frontend/services/gemini.ts:13-17	하객 이름을 그대로 프롬프트에 삽입. 축하말 버튼 반복 클릭 제한 없음(비용)
13	낮음	품질	frontend/constants.ts:5-30,49-62, frontend/index.html:10-12	예식 데이터, OG 메타(이름 하드코딩), picsum 갤러리, 외부 샘플 MP3 가 코드에 박혀 있어 커플마다 코드 수정 필요. constants.ts:27 mapUrl 은 미사용
14	낮음	성능	frontend/index.html:14	Tailwind CDN 런타임(프로덕션 비권장)
15	낮음	품질	frontend/components/Hero.tsx:50, frontend/components/Guestbook.tsx:19	시간 문자열 "PM 12:00" 하드코딩(date 값과 불일치 가능), 사용하지 않는 scrollRef

심각도 높음 상세

#1 방문마다 Imagen 생성. Hero 와 Location 이 useEffect 에서 매 마운트마다 generateHeroImages(3), generateMapImage() 를 호출한다. 청첩장은 하객 수백 명이 여러 번 여는 페이지라 이미지 생성 비용이 하객 수에 비례해 늘고, rate limit(server.js:37-46, IP당 15분 100회)은 비용 방어가 되지 못한다. 이미지는 커플당 1회만 만들면 되므로 관리자 생성 단계에서 만들어 스토리지에 저장하고 하객 페이지는 정적 URL 만 읽어야 한다.

#2 실제 커플/실제 지도가 아님. 청첩장의 핵심 가치는 커플 본인의 사진과 정확한 위치 안내인데, 현재 히어로는 프롬프트로 만든 가상 인물이고 지도는 "Naver Map style 스크린샷"을 Imagen 이 상상해 그린 이미지다. 하객이 이 이미지를 보고 길을 찾으면 오도될 수 있다. 히어로는 업로드한 사진(선택적으로 AI 보정), 지도는 카카오/네이버 정적 지도 API 또는 임베드로 교체해야 한다.

#3 배포 불가능한 AI 호출 경로. 인터셉터와 Vite 개발 서버 프록시에 의존하는 구조라 vite build 결과물을 정적 호스팅하면 /api-proxy 가 없어 AI 기능이 모두 폴백으로 떨어진다. 반대로 백엔드를 공개하면 프론트에 박힌 PROXY_HEADER(#5)만 알면 누구나 임의 모델/임의 프롬프트를 이 GCP 프로젝트 비용으로 호출할 수 있다(#4). AI 는 "축하말 생성" 한 가지 용도 전용 엔드포인트(입력 길이 제한, 고정 모델, 고정 프롬프트)로 좁히고 범용 프록시는 제거해야 한다.

4. 효율성 개선 제안

- 구조/아키텍처: 하객용 페이지는 정적 사이트(constants 대신 JSON/DB 의 invitation 레코드 1건 렌더링)로, AI 는 서버리스 함수 1~2개(/api/congrats, 관리자용 /api/hero-image)로 분리한다. 범용 Vertex 프록시, 인터셉터, WebSocket 프록시(BidiGenerateContent, 미사용)를 제거한다. 장기적으로 /i/:slug 라우트로 여러 커플을 한 코드베이스에서 서비스하는 템플릿 구조가 상품화의 전제다.
- 성능: 이미지 생성 결과를 Supabase Storage 등에 캐시해 하객 방문 시 AI 호출 0건으로 만든다. Tailwind 를 @tailwindcss/vite 빌드로 전환하고 갤러리 이미지에 loading="lazy" 와 적정 해상도를 지정한다.
- 개발 생산성: tsconfig.json + @types/react 추가, tsc --noEmit 스크립트, 의존성을 lock 기준 캐럿 범위로 고정, backend 에 node-fetch 명시(또는 내장 fetch + Readable.fromWeb 로 교체).
- 운영/배포: Supabase URL/anon key 를 VITE_ 환경변수로 이동하고 guestbook_messages 테이블 마이그레이션 SQL 과 RLS(INSERT 는 길이 제한 체크, DELETE 는 관리자만)를 저장소에 포함. Vercel 등 정적 호스팅 + 서버리스로 배포 경로를 문서화. OG 메타를 데이터 기반으로 생성.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	Hero/Location 의 런타임 Imagen 호출 제거. 히어로는 constants(이후 DB)의 사진 URL 배열, 지도는 카카오 지도 임베드 또는 정적 지도 이미지로 교체	P0	S	Sonnet 5.5	호출부 삭제와 이미지 소스 치환 위주
2	범용 Vertex 프록시/인터셉터/WS 프록시 제거, 축하말 전용 서버리스 엔드포인트(고정 모델, 이름 20자·결과 길이 제한, IP rate limit, 서버 측 시크릿) 신설	P0	M	Fable 5.1	비용·남용 방어가 걸린 보안 경계 재설계
3	카카오 SDK SRI 해시를 공식 값으로 교체(또는 integrity 제거), JS 키를 VITE_KAKAO_JS_KEY 환경변수로 이동	P0	S	Sonnet 5.5	설정 수정
4	tsconfig.json, @types/react·react-dom 추가, typecheck 스크립트, 의존성 latest 를 lock 기준 버전으로 고정, backend 에 node-fetch 명시	P0	S	Sonnet 5.5	설정 보일러플레이트
5	Supabase 연동 정리: URL/키 환경변수화, 테이블·RLS 마이그레이션 SQL 추가, insert 오류 처리, relation 저장, id 는 DB 기본값 사용, 삭제 버튼은 관리자만 노출	P1	M	Opus 5.5	2~4개 파일 API 연동 + RLS 설계
6	방명록 입력 검증(이름/내용 길이, 공백 차단, 연속 등록 쿨다운)과 오류 토스트	P1	S	Sonnet 5.5	단순 폼 로직
7	WEDDING_DATA·갤러리·음악·OG 메타를 단일 invitation JSON 스키마로 분리하고 하드코딩 시간 문자열(Hero.tsx:50) 제거	P1	M	Opus 5.5	타입/데이터 모델 정리, 여러 컴포넌트 수정
8	index.html 폰트 오타 수정, Tailwind CDN -> 빌드 전환, 미사용 scrollRef/mapUrl 정리	P1	S	Sonnet 5.5	스타일/린트 정리
9	CLAUDE.md, README 를 실제 구조(정적 + 서버리스, 환경변수 목록, 배포 절차)로 갱신	P1	S	Sonnet 5.5	문서화
10	/i/:slug 라우팅 + Supabase invitations 테이블로 다중 커플 지원 구조 설계 및 골격 구현	P2	L	Fable 5.1	데이터 모델과 라우팅 재설계, 상품화 핵심
11	관리자 편집 화면(예식 정보 입력, 사진 업로드 to Storage, 선택적 AI 히어로 보정 1회 생성 후 저장)	P2	L	Opus 5.5	범위가 명확한 기능 구현
12	갤러리 라이트박스, 이미지 lazy loading, 음악 파일 자체 호스팅	P2	S	Sonnet 5.5	UI 반복 작업

6. 종합 평가

- 강점 3개
  1. 섹션별 컴포넌트 분리가 깔끔하고 각 파일이 200줄 이하라 이해와 수정이 쉽다. 거대 단일 컴포넌트 문제가 없다.
  2. 계좌 복사, tel/sms 링크, 네이버/카카오 지도 딥링크, 달력 등 한국 모바일 청첩장의 필수 UX 가 이미 갖춰져 있고 모션 완성도가 높다.
  3. API 키를 브라우저에 두지 않고 백엔드 ADC 로 인증하는 방향 자체는 옳다(vite.config.ts:8 의 API_KEY 는 더미). Supabase 미설정 시 localStorage 폴백으로 데모가 깨지지 않는다.
- 약점 3개
  1. AI 사용 방식이 제품 요구와 반대다. 하객 방문마다 비싼 이미지 생성을 하고, 결과물이 실제 커플/실제 지도가 아니다.
  2. 개발용 범용 Vertex 프록시 구조에 묶여 공개 배포가 불가능하고, 억지로 공개하면 오픈 프록시가 된다.
  3. 데이터가 단일 커플 하드코딩이고 Supabase/카카오가 placeholder 라 "상품"이 아닌 "1인용 데모" 상태다. 타입 검사와 버전 고정도 없다.
- 모델별 비율: Fable 17% (2/12) / Opus 25% (3/12) / Sonnet 58% (7/12)
- 권장 진행 순서
  1. 1단계(P0): 런타임 Imagen 제거, 범용 프록시를 축하말 전용 엔드포인트로 교체, 카카오 SDK/설정/타입 정비 -> 배포 가능한 정적 청첩장.
  2. 2단계(P1): Supabase 방명록 실연동과 RLS, invitation 데이터 스키마 분리, 스타일/문서 정리.
  3. 3단계(P2): 다중 커플 라우팅, 관리자 편집 화면, 갤러리 개선 -> 판매 가능한 템플릿.
- 최종 권고: **계속 발전**. nexus-micro-web-builder 가 이 프로젝트를 상품화 대상(모델 0번)으로 지목하고 있고, UI 와 섹션 구성은 이미 재사용 가치가 높다. 다만 현재 AI 구조는 버리고 "정적 청첩장 + 최소 서버리스" 로 다시 세우는 것이 전제이며, 1단계 완료 후 이 코드를 청첩장/돌잔치/부고장 등 다른 마이크로 웹의 공통 템플릿 베이스로 삼는 것을 권한다.
- 주 담당 모델: **Opus 5.5**. 작업 수로는 Sonnet 비중이 가장 크지만 대부분 S 규모 정리 작업이고, 이 프로젝트의 가치를 결정하는 것은 Supabase 연동, 데이터 스키마 분리, 관리자 화면처럼 2~5개 파일에 걸친 명확한 중간 규모 기능이다. 비용·남용 방어가 걸린 프록시 교체(작업 2)와 다중 커플 구조 설계(작업 10) 두 건만 Fable 5.1 에 맡기고, 나머지 흐름은 Opus 5.5 가 주도하면서 단순 정리 작업을 Sonnet 5.5 로 넘기는 구성이 가장 효율적이다.